

AttentionOS

Персональная система наблюдения за рабочим состоянием

Идея в одной фразе

Система локально наблюдает за рабочими паттернами пользователя, учит его персональный baseline, оценивает вероятность падения эффективности и предлагает проверяемые вмешательства - например, перерыв - только когда это оправдано данными.

Формат: pet project / open-source portfolio project

Цель: показать полный цикл ML-продукта - telemetry → dataset → feature engineering → персональная модель → intervention → измерение эффекта.

Статус документа: техническое задание и архитектурный концепт

Версия: 0.1 | Дата: август 2026

1. Концепция проекта

AttentionOS - desktop-приложение, которое не ограничивается учётом времени. Его задача - построить персональную модель работоспособности пользователя: понять, как у конкретного человека меняются концентрация, стабильность работы и вероятность ошибок в зависимости от длительности сессии, времени суток, количества переключений, недавней нагрузки и реакции на перерывы.

1.1. Проблема

- Обычные time trackers отвечают на вопрос «куда ушло время», но слабо отвечают на вопрос «когда продолжение работы уже невыгодно».
- Фиксированные Pomodoro-интервалы одинаковы для всех и не учитывают персональные паттерны.
- Человек часто замечает переутомление уже после заметного падения качества работы.
- Самооценка состояния субъективна; полезнее сопоставлять её с объективными поведенческими признаками.

1.2. Главная гипотеза

Гипотеза H1

По пассивной телеметрии рабочего поведения и коротким self-report меткам можно построить персональную модель, которая лучше простых правил предсказывает периоды низкой эффективности.

Гипотеза H2

Для конкретного пользователя эффект перерыва зависит от текущего состояния; система может постепенно оценивать, когда перерыв действительно улучшает последующую работу.

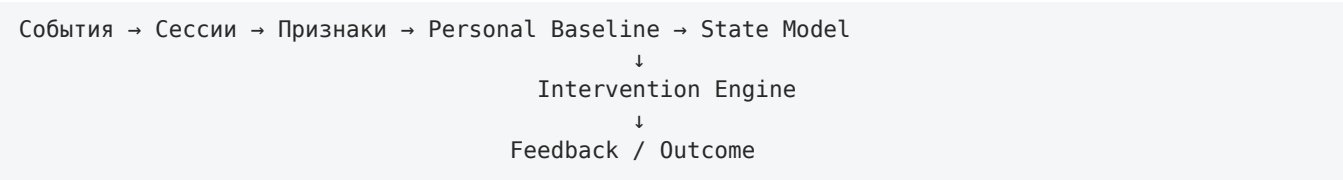
1.3. Чем проект НЕ является

- Не медицинский продукт и не система диагностики выгорания, СДВГ, депрессии или иных состояний.
- Не keylogger: содержание нажатий клавиш не сохраняется.
- Не employee surveillance: первая версия предназначена для самого пользователя.
- Не «LLM-обёртка». LLM может объяснять агрегированные отчёты, но не является ядром продукта.

2. Пользовательский сценарий

1. Пользователь запускает AttentionOS в фоне.
2. Collector фиксирует безопасную телеметрию: активное приложение, длительность активности, idle, переключения и агрегированную интенсивность ввода.
3. Пользователь при желании отмечает текущий тип задачи: Coding, ML, Math, English, Rest и т.д.
4. Несколько раз в день приложение просит оценку субъективной эффективности/усталости по шкале от 1 до 5.
5. Feature pipeline строит признаки по последним 15/30/60/120 минутам.
6. После накопления данных персональная модель оценивает текущее состояние и прогноз на ближайший рабочий интервал.

7. Intervention engine предлагает действие только при достаточной уверенности: продолжить, сделать короткий перерыв, сменить тип задачи или завершить сессию.
8. После вмешательства система измеряет, улучшились ли признаки и self-report, и обновляет персональную статистику.



2.1. Пример результата

Последние 52 минуты:

- context switches: +71% к твоему baseline

- средняя непрерывная сессия: 4.2 мин вместо 11.8 мин

- idle bursts: +38%

- self-report за похожие периоды: 2.1 / 5

Оценка: высокая вероятность снижения эффективности.

Рекомендация: 18-25 минут вне рабочего контекста.

Уверенность модели: 0.78

3. Функциональные требования

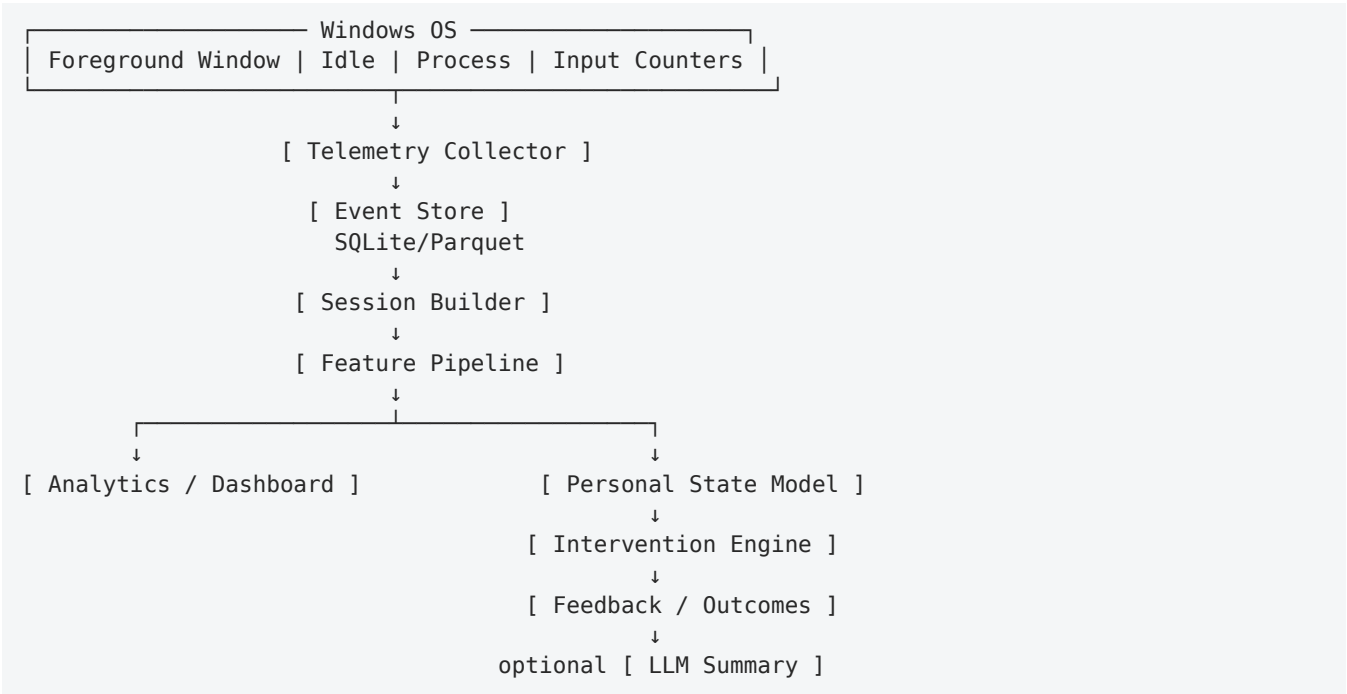
ID	Функция	Требование	Приоритет
F-01	Foreground tracking	Определять активный процесс и длительность активности с шагом 2-5 секунд.	MUST
F-02	Idle tracking	Определять периоды отсутствия пользовательского ввода через WinAPI.	MUST
F-03	Context switches	Фиксировать смену активного приложения и вычислять частоту переключений.	MUST
F-04	Input activity	Сохранять только количество событий клавиатуры/мыши за окно времени; не сохранять содержимое ввода.	SHOULD
F-05	Task labels	Позволять вручную отметить категорию текущей задачи.	SHOULD
F-06	Self-report	Запрашивать 1-5 оценку эффективности/усталости с настраиваемой частотой.	MUST

ID	Функция	Требование	Приоритет
F-07	Timeline	Показывать дневную ленту приложений, focus-блоков, idle и task labels.	MUST
F-08	Daily report	Строить агрегированный отчёт по дню и сравнение с baseline.	MUST
F-09	State model	После накопления датасета выдавать персональную оценку состояния/эффективности.	SHOULD
F-10	Interventions	Предлагать перерыв/продолжение/смену задачи и логировать реакцию пользователя.	SHOULD
F-11	AI summary	Опционально генерировать текстовый summary только из агрегированных метрик.	COULD
F-12	Export	Экспорт CSV/Parquet с обезличенной телеметрией и фичами.	SHOULD

4. Нефункциональные требования

Область	Требование
Privacy	Local-first. По умолчанию данные хранятся локально. Реальные пользовательские данные не попадают в Git-репозиторий.
Performance	Collector должен потреблять минимальный CPU/RAM и не мешать работе; целевой overhead CPU < 1-2% в фоне.
Reliability	Потеря UI не должна останавливать сбор событий; collector и UI логически разделены.
Security	SQLite/Parquet-файлы хранятся в user data directory. Секреты и API-ключи - только через env/keyring.
Explainability	Каждый alert должен показывать основные признаки, повлиявшие на решение, а не только score.
Reproducibility	Фиксированные random seeds, versioned feature schema, training config и ML metrics.
Portability	Первая версия - Windows 10/11. Архитектура collector должна позволять позже добавить macOS/Linux adapters.
Ethics	Никаких выводов о медицинском состоянии. Только продуктивность и наблюдаемые паттерны.

5. Архитектура



5.1. Модули

Модуль	Ответственность	Примечание
collector	Сбор foreground/idle/input counters/process metadata.	Никакого содержимого текста.
storage	SQLite schema, migrations, retention, export.	Raw events + derived sessions.
sessions	Склейка событий в рабочие блоки и классификация idle/focus.	Правила versioned.
features	Rolling window features 15/30/60/120 мин.	Отдельная feature schema.
model	Baseline + supervised/personalized model.	Сначала простые модели.
interventions	Правила показа рекомендаций и outcome logging.	Не спамить пользователя.
ui	Timeline, отчёт, feedback, model explanation.	MVP: Streamlit; позже PySide6.
ai_summary	Необязательное LLM-объяснение агрегатов.	Без передачи raw window titles по умолчанию.

6. Данные

6.1. Raw event schema

```
activity_event
- ts_start: datetime
- ts_end: datetime
- process_name: str
- window_title_hash / normalized_title: optional
- idle_seconds: float
- keyboard_events: int
- mouse_events: int
- task_label: optional[str]
- collector_version: str
```

6.2. Self-report schema

```
self_report
- timestamp: datetime
- perceived_effectiveness: int # 1..5
- perceived_fatigue: int # 1..5
- task_difficulty: int # optional 1..5
- note: optional[str]
```

6.3. Intervention schema

```
intervention
- timestamp: datetime
- type: break | continue | switch_task | stop
- predicted_state: float
- confidence: float
- accepted: bool
- duration_minutes: optional[int]
- outcome_window_minutes: int
- post_state_delta: optional[float]
```

Правило публикации

В публичный репозиторий входят только synthetic/demo data и скрипты генерации. Личные raw events и реальные названия окон добавляются в .gitignore.

7. ML-часть

7.1. Базовые признаки

Группа	Примеры признаков
Focus	mean_focus_block, max_focus_block, uninterrupted_ratio
Switching	switches_15m, switches_60m, unique_apps, switch_entropy

Группа	Примеры признаков
Idle	idle_ratio, idle_bursts, time_since_last_break
Input dynamics	keyboard_rate, mouse_rate, rate_change_vs_baseline
Temporal	hour_of_day, weekday, session_age, work_since_day_start
Workload	active_minutes_2h, active_minutes_day, previous_session_length
Personal baseline	z-score/percentile относительно аналогичного времени и типа задачи
Task	task_label, task difficulty, historical performance for task type

7.2. Последовательность моделей

1. Baseline rules: rolling z-scores и percentile-based alerts. Нужны как понятный benchmark.
2. Supervised baseline: Logistic Regression / RandomForest / CatBoost на self-report labels.
3. Персональная модель: обучение только на одном пользователе или глобальная модель + user-specific calibration.
4. Позже: temporal model (LightGBM по lag-фичам, HMM/state-space model, TCN/Transformer только если данных действительно достаточно).

Анти-паттерн

Не начинать с нейросети. Главная сложность проекта - корректный сбор данных, target definition и персонализация, а не выбор самого модного алгоритма.

7.3. Targets

- Regression: predicted perceived_effectiveness $\in [1, 5]$.
- Classification: low-performance state = effectiveness ≤ 2 .
- Intervention outcome: improvement after break / no improvement.
- Дополнительно: прогноз вероятности ухудшения в ближайшие 30-60 минут.

7.4. Метрики

Задача	Метрики	Комментарий
Regression	MAE, Spearman correlation	Корреляция важна из-за субъективной шкалы.
Classification	ROC-AUC, PR-AUC, F1, calibration	PR-AUC важнее при редких low-state.
Intervention	Δ self-report, Δ behavioral score	Сравнивать accepted intervention с похожими baseline периодами.
UX	alert acceptance rate, dismiss rate	Если alerts игнорируются - модель/порог плохие.

8. Intervention Engine

Первая версия не должна автоматически прерывать работу. Она предлагает действие и собирает обратную связь. Решение строится на score, uncertainty и cooldown.

```
if risk_low_performance > 0.75 and confidence > 0.65:
    if minutes_since_last_alert > cooldown:
        recommend_break(duration=20)
else:
    no_action()
```

- Порог настраиваемый и персональный.
- После отклонённого alert система не повторяет его сразу.
- Перерыв не считается «успешным» автоматически - требуется измерить post-window.
- Для экспериментов хранить reason codes: switching, long_session, unusual_idle, baseline deviation.

9. План реализации

Этап	Что сделать	Критерий готовности
v0.1 Telemetry	Foreground, idle, context switches, SQLite, дневной timeline.	7 дней данных без пропусков > 5 мин и без заметного overhead.
v0.2 Sessions	Task labels, session builder, daily analytics, CSV/Parquet export.	Можно восстановить день и получить агрегаты по задачам.
v0.3 Labels	Self-report UI, reminders, dataset builder.	≥ 50 размеченных состояний.
v0.4 Personal Model	Baseline + CatBoost/LogReg, evaluation, explanations.	Модель стабильно превосходит naive baseline на holdout.
v0.5 Interventions	Break recommendations, cooldown, outcome logging.	Есть измеряемый post-intervention dataset.
v1.0 Portfolio Release	Installer, docs, synthetic demo dataset, screenshots, reproducible training.	Новый пользователь может установить и получить первый отчёт без ручной настройки кода.

10. Рекомендуемая структура репозитория

```
attentionos/
├── src/attentionos/
│   ├── collector/
│   │   ├── foreground.py
│   │   ├── idle.py
│   │   └── input_activity.py
│   ├── storage/
│   │   ├── db.py
│   │   └── schema.py
│   ├── sessions/
│   ├── features/
│   ├── models/
│   ├── interventions/
│   ├── ui/
│   └── config.py
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_analysis.ipynb
│   └── 03_modeling.ipynb
├── tests/
├── scripts/
│   ├── generate_demo_data.py
│   └── train_model.py
├── data/demo/
├── docs/
├── pyproject.toml
├── .gitignore
├── LICENSE
└── README.md
```

11. Технологический стек

Компонент	Рекомендация	Почему
Language	Python 3.12+	Основной стек проекта и ML.
Windows telemetry	pywin32 + ctypes + psutil	Foreground window, GetLastInputInfo, process metadata.
Storage	SQLite + SQLAlchemy/SQLModel	Надёжно, локально, просто мигрировать.
Analytics	pandas / polars, numpy	Feature engineering и отчёты.
ML	scikit-learn + CatBoost	Сильные baselines на табличных данных.
Explainability	SHAP / permutation importance	Показать, почему модель сработала.
UI MVP	Streamlit	Быстрый timeline/dashboard.
Desktop later	PySide6	Нативное приложение и tray.
Plots	Plotly или matplotlib	Timeline, rolling metrics, model diagnostics.

Компонент	Рекомендация	Почему
Testing	pytest	Collector/session/feature regression tests.
Packaging	uv/Poetry + PyInstaller	Повторяемая установка и Windows executable.
Optional AI	LLM API или local model	Только summary агрегированных показателей.

12. Критерии готовности рет-проекта

- Репозиторий запускается по README на чистой Windows-машине.
- Collector стабильно работает в фоне ≥ 8 часов.
- Есть минимум неделя реальных данных автора, но они не опубликованы.
- Есть synthetic/demo dataset, на котором воспроизводятся графики и обучение.
- Есть EDA: распределение focus blocks, context switches, time-of-day effects, relation with self-report.
- Есть минимум один простой baseline и одна ML-модель с честным temporal holdout.
- Есть model card: target, features, limitations, leakage risks, metrics.
- Есть UI с timeline, day summary и объяснением хотя бы одного alert.
- Есть видео/GIF demo 30-90 секунд в README.
- Есть раздел Privacy: что собирается, что не собирается, где хранится и как удалить данные.

13. Тестирование

Тип	Что проверять
Unit	Session merge/split, idle thresholds, rolling feature calculations, timezone handling.
Integration	Collector $\rightarrow$ DB $\rightarrow$ sessions $\rightarrow$ features без потери событий.
ML	Temporal split, leakage tests, stability across days, calibration.
Performance	CPU/RAM overhead collector в течение 8-часового запуска.
Privacy	Проверка, что raw keystrokes и чувствительный текст не записываются.
Failure modes	Sleep/wake, app crash, clock change, multiple monitors, UAC/system windows.

14. Главные риски и ограничения

Риск	Почему важен	Как снизить
Слабый target	Self-report шумный и субъективный.	Несколько вопросов, повторные измерения, behavioral proxy targets.
Мало данных	Один пользователь быстро даёт мало размеченных точек.	Собирать события часто, labels реже; начать с simple models.

Риск	Почему важен	Как снизить
Confounding	Сложность задачи влияет на поведение сильнее усталости.	Task label, difficulty, сравнение внутри типов задач.
Data leakage	Rolling features могут случайно использовать будущие события.	Строго causal feature generation и temporal split.
Notification fatigue	Плохие alerts будут раздражать.	Cooldown, uncertainty threshold, opt-in interventions.
Privacy	Window titles могут содержать личные данные.	По умолчанию process-only; titles optional/hashed/local.
Overclaiming	Нельзя делать медицинские выводы.	Язык «рабочие паттерны/эффективность», explicit disclaimer.

15. Как оформить для портфолио

Главная ценность проекта для портфолио - не количество экранов, а демонстрация инженерного и исследовательского цикла.

Что показать	Что должен увидеть reviewer
README hero	1 GIF + одна фраза проблемы + одна схема архитектуры.
Data pipeline	Как безопасно собираются события и строятся сессии/фичи.
ML experiment	Почему target такой, какие baselines, temporal validation, ограничения.
Personalization	Сравнение global rule vs personal baseline/model.
Intervention	Alert → accepted/rejected → outcome measurement.
Engineering	Tests, configs, migrations, packaging, CI.
Privacy	Local-first design и synthetic demo data.

Позиционирование в README

AttentionOS is a local-first personal performance modeling system. It learns an individual behavioral baseline from desktop telemetry and self-reports, predicts low-performance states, and evaluates whether adaptive interventions actually help.

16. Definition of Done для первой публичной версии

- [] Установка на Windows одной командой / через release
- [] Collector работает 8+ часов без падений
- [] Timeline и daily report
- [] Self-report labels
- [] Reproducible feature pipeline
- [] Baseline + ML model
- [] Temporal validation
- [] Alert explanation
- [] Intervention logging
- [] Synthetic demo dataset
- [] Unit + integration tests
- [] Privacy documentation
- [] GIF/video demo
- [] README с архитектурой и результатами

17. Что НЕ делать в первой версии

- Не делать мобильное приложение.
- Не делать командный/корпоративный режим.
- Не интегрировать десятки календарей, мессенджеров и wearable API.
- Не строить deep learning до появления достаточного датасета.
- Не делать marketplace, подписки, Stripe и коммерческий backend.
- Не отправлять raw telemetry в облако ради AI-summary.
- Не обещать «детекцию выгорания».

18. Первый спринт: конкретное ТЗ на 7 дней

День	Результат
1	Скелет проекта, config, SQLite schema, foreground process collector.
2	Idle detection + event batching + корректное завершение/возобновление после sleep.
3	Session builder и context-switch metrics.
4	Простой Streamlit timeline и статистика дня.
5	Task labels + self-report form.
6	Feature pipeline для 15/30/60 минут + demo dataset generator.
7	README, screenshots, 8-часовой soak test, исправление багов.

Milestone после 7 дней

Приложение уже должно быть полезно без ML: оно честно восстанавливает рабочий день и начинает собирать датасет, на котором можно проверять главную гипотезу проекта.